VIEWS

Delivery Note

Nine Deep-Learning Forecast Models

— ◇ —

Identifier	`r2darts2-pgm-global-calibration-2026-10`
Checksum	`60597598`
Date	10-10-2026
Author	Simon Polichinel von der Maase

Scope *what the data is*

Partition	Calibration
Spatial resolution	PRIO-GRID cell
Spatial extent	Global land — 64,818 cells
Temporal resolution	Month
Temporal extent	Forecasts Jan 2018 to Dec 2021; trained on Jan 1990 to Dec 2017
Forecast structure	13 rolling origins, each forecasting 36 months
Targets	State-based, non-state and one-sided fatalities

Provenance *what produced it*

Model library	views-r2darts2 0.2.4, wrapping Darts (`unit8co/darts`)
Framework	views-pipeline-core 3.3.4
Data backend	views-datafactory 1.13.0
Source commit	`views-models@be7b7b6`
Computed on	Rented cloud GPUs (RunPod)

PRIO – Oslo

VIEWS
PREDICTING CONFLICT & HUMANITARIAN IMPACTS

PRIO

1 Context

This Delivery Note accompanies `r2darts2-pgm-global-calibration-2026-10`, a set of conflict forecasts produced by nine deep-learning models on the calibration partition at global PRIO-GRID month. It is the authoritative description of what was handed over: the data itself carries no documentation, and the folder is not self-explanatory.

The purpose of this note is to:

- record what was delivered, in enough detail to identify it again later;
- describe the models that produced it, and what distinguishes them from one another;
- state how the files are organised and how they are intended to be consumed; and
- set out what is *not* in the delivery, and what is not yet known about it.

This note covers:

- the nine models, their architectures, and the published work they come from (Sections 2–3);
- the objective functions they were trained on, including one written in-house (Section 4);
- the file layout, schema and units of the delivered forecasts (Section 5);
- how the delivery is consumed, by the ensemble-construction tool or by hand (Section 6); and
- what the delivery excludes, and the caveats that bear on using it (Sections 7–8).

This note explicitly does *not* cover:

- **any evaluation of the models.** Nothing here has been scored, ranked or compared. No metric has been computed beyond a check that the files are not corrupt;
- any recommendation as to which models to prefer, or whether to use them at all;
- the validation or forecasting partitions, or any live forecast; and
- the internal machinery that produced the run — what it took to compute these forecasts is recorded separately, in `reports/postmortem_darts_calibration_2026-10.md`.

The delivery is a fixed artefact. It is identified by the identifier and checksum on the title page — quote both — and is not revised in place: any correction is issued as a new delivery with a new identifier and a new checksum.

2 The Models

All nine are **off-the-shelf deep-learning models for time series**, taken from the Python library **Darts** (`unit8co/darts`). The architectures are the library's, unmodified — each is a

published model used as Darts implements it. What we supply is the data, the configuration, and for six of the nine a loss function of our own (Section 4).

They are *global* models in the forecasting sense: a single set of parameters is pooled across all 64,818 cells. The closest familiar analogue is a pooled panel estimator — a cell with almost no conflict history is still forecast using structure learned from the whole grid. Each cell's own level and scale are restored separately, by the normalisation described in Section 3.

Table 1: The nine delivered models.

Model	Type	Loss	Predictors	Hyperparameters	Features
dark_river	N-BEATS	MSE	Outcomes only	link	link
old_rules	N-BEATS	Spotlight	Outcomes only	link	link
blue_ocean	N-BEATS	Spotlight	+ 69 covariates	link	link
silent_fox	TSMixer	MSE	Outcomes only	link	link
dancing_monkey	TSMixer	Spotlight	Outcomes only	link	link
brave_heart	TSMixer	Spotlight	+ 69 covariates	link	link
red_hawk	TiDE	MSE	Outcomes only	link	link
dark_necessities	TiDE	Spotlight	Outcomes only	link	link
golden_eagle	TiDE	Spotlight	+ 69 covariates	link	link

The link columns open each model's configuration files on GitHub, fixed at the delivery commit on the title page: **Hyperparameters** is `config_hyperparameters.py`, which sets the architecture, the loss and the training schedule; **Features** is `config_queryset.py`, which declares the predictors the model was given.

The names are arbitrary labels, assigned according to the `adjective_noun` convention used across views-models, and carry no meaning.

Within each block of three, exactly two settings vary — the loss function, and whether the model sees covariates. Everything else is identical: layer widths, depth, training epochs, learning rate, early stopping. So inside a block, each row differs from the one above it in a single setting. Across blocks the architecture changes and the comparison is a different one.

- **Outcomes only** — the model's inputs are the past values of the three outcomes and nothing else. Purely autoregressive. Six models: the first two rows of each block.
- **+ 69 covariates** — the same three histories plus 69 predictors: ACLED event and fatality counts, land cover shares, population and built-up area, infant and child mortality, V-Dem governance indices, SHDI development indices, travel time to cities, mineral and petroleum deposits, growing season and harvested area, and terrain. Three models: the third row of each block.

Note that the two splits are not the same split. Six models are outcomes-only and six use the Spotlight loss, but they are different sixes: `blue_ocean`, `brave_heart` and `golden_eagle` have covariates *and* Spotlight, while `dark_river`, `silent_fox` and `red_hawk` have neither.

There is no model combining MSE with the covariates. Three configurations exist, not four.

Covariate Preparation

Covariates are prepared in two stages, by different parties and to different standards.

Within the datafactory, preparation is specified. Annual sources are extended across their twelve months by per-source rules; values outside a source's observed range are manufactured, zero before the first observation and the last value held after it; absent coverage is filled with zeros rather than nulls; raster and polygon sources are attached to grid cells by aggregation and spatial join. Feature engineering is excluded by design, cross-feature derivation being assigned to the model layer. Each of these is a recorded decision with an architecture decision record behind it — ADR-014, ADR-042, ADR-047 and ADR-052 in the datafactory repository, with a documentation card per source. That is the authoritative account and this note does not restate it.

Preparation after the fetch is set in the model configuration, and is documented only by the configuration itself. Two operations apply here: the selection of which covariates each model receives, and an `asinh` transform applied to all 72 columns, the three outcomes and the 69 covariates alike, before training. Both are visible in the files linked in Table 1. Neither carries a decision record or a written rationale, and whether either is warranted is not established by this note.

Covariate Coverage

The datafactory publishes, for each source, the month range in which its observations are real. Values outside that range exist but are manufactured. Checked against this delivery's partition, four of the five sources cover it fully and one does not.

Table 2: Source coverage against the calibration partition.

Source	Observed from	Training 1990–2017	Test 2018–2021
ACLED (7 covariates)	January 2020	**none — all zeros**	real from January 2020
V-Dem	January 1989	real	real
SHDI	January 1989	real	real
Population	January 1989	real	real
Built-up surface	January 1989	real	real

The seven ACLED covariates are therefore zero for every month these models were fitted on, and become non-zero partway through the forecast window. This is a sharper problem than an absent feature: a predictor that is constant in training and varies in test is one the model has learned to disregard. Nor is it visible in the data, since manufactured and observed zeros are written identically.

The coverage bounds are published so that manufactured values can be identified, and were available when these models were configured; the omission is downstream of the datafactory, not

within it. Three models are affected — `blue_ocean`, `brave_heart` and `golden_eagle`. The six outcomes-only models use no covariates.

3 Model Architectures

All three are feed-forward neural networks built from ordinary dense layers. None uses recurrence, convolution or attention.

N-BEATS

A deep stack of fully connected layers with backward and forward residual links. It builds the forecast as a sum of learned basis functions, which is what makes it partly interpretable. Designed for univariate point forecasting and benchmarked on the M3, M4 and TOURISM competition datasets.

> Darts: `NBEATSModel`
> Paper cited there: Oreshkin, B. N., Carpov, D., Chapados, N. & Bengio, Y. (2020), *N-BEATS: Neural basis expansion analysis for interpretable time series forecasting*, ICLR 2020. arXiv:1905.10437

TSMixer

Dense layers only, applied alternately along the time axis and across the predictor variables, so the model mixes temporal structure and cross-variable structure in turn. Built for the multivariate case.

> Darts: `TSMixerModel`
> Paper cited there: Chen, S.-A., Li, C.-L., Yoder, N., Arik, S. O. & Pfister, T. (2023), *TSMixer: An All-MLP Architecture for Time Series Forecasting*, Transactions on Machine Learning Research. arXiv:2303.06053

TiDE

An encoder–decoder, again dense layers only. It compresses the history and the predictors into a single code, then decodes the whole 36-month path from it in one pass.

> Darts: `TiDEModel`
> Paper cited there: Das, A., Kong, W., Leach, A., Mathur, S., Sen, R. & Yu, R. (2023), *Long-term Forecasting with TiDE: Time-series Dense Encoder*. arXiv:2304.08424

One Setting All Nine Share

All nine use **reversible instance normalisation**: each cell's series is standardised before the network sees it and the transformation is inverted on the way out. The model therefore fits on a per-cell normalised scale, while the forecasts you have are in counts.

> Paper cited by Darts for all three models: Kim, T., Kim, J., Tae, Y., Park, C., Choi, J.-H. & Choo, J. (2022), *Reversible Instance Normalization for Accurate Time-Series Forecasting against Distribution Shift*, ICLR 2022.

4 Loss Functions

MSE — ordinary squared error, every cell-month weighted equally. The library's own implementation, a thin wrapper around PyTorch's. Used by `dark_river`, `silent_fox` and `red_hawk`.

Spotlight (`SpotlightLossLogcosh`) — in-house, not off the shelf. A three-part objective computed after an `asinh` transform. It scores the *shape* of the error only in cells where the observation or the forecast exceeds one fatality; separately penalises the *level* in cells whose observation exceeds one fatality; and separately penalises forecasting conflict in cells that stayed below it. It uses `log cosh` rather than squared error, so a single very large observation does not dominate the gradient. Used by the other six.

Development and Validation Status

Spotlight is not a one-off function but the production member of a sustained line of work. The first variant was committed in March 2026 and the one used here at the end of April; development continued into late August. The family now comprises six Spotlight variants and three related objectives — roughly 2,250 lines across nine modules — attracting some 216 commits over six months. The library's own documentation sets out five earlier generations of the design, recording for each what it addressed and what it exposed. By any ordinary measure this represents a considerable investment of development effort.

Of the objective's two parameters, one is derived rather than fitted: the threshold separating event from quiet cells is $\operatorname{asinh}(1) \approx 0.88$, exactly one fatality on the transformed scale. The other, a weight on a spectral term, is set by hyperparameter sweep.

What the record does not contain is any formal account of it. No citation appears in the source, its documentation, the commit history or the architecture decision records; two other objectives in the same library do cite published work, so the omission is particular to this family rather than a habit of the codebase. Neither is it covered by tests. The library's suite exercises eight of its objectives and this is not among them, so nothing checks its gradients, compares it against a reference implementation, or establishes that the code computes what the written account describes — this for the objective on which 37 of the platform's 42 darts models are trained.

Six months of concentrated development across five successive generations is not the profile of an untried design, and the written account of its construction is detailed. It is reasonable to infer that the objective has been exercised extensively, if informally, during that period. Informal exercise is not validation, however, and the distinction is what this note is obliged to record: what is absent is not the work but the evidence of it. A derivation a third party can follow, and tests that fail when the implementation is wrong, remain outstanding and should be produced for an objective in production use at this scale.

On that basis this note takes no position on whether the objective is sound. It may be excellent, adequate, or quietly wrong, and nothing in the available record distinguishes those possibilities. Six of the nine delivered models were trained on it and three on ordinary squared error, which bears on how much weight a comparison between those two groups can carry.

5 The Delivered Files

One folder per model. Inside each: thirteen Parquet files, `..._00` through `..._12`, and a short text file recording when the model ran, which version of the software produced it, and which configuration it used.

The thirteen files are thirteen forecast origins, stepping forward one month at a time. An origin is the last month the model was allowed to see; from there it forecasts the next 36 months.

Table 3: The thirteen forecast origins.

File	Last month seen	Months forecast
`..._00`	December 2017	January 2018 – December 2020
`..._01`	January 2018	February 2018 – January 2021
	...	
`..._12`	December 2018	January 2019 – December 2021

Within every file the earliest month is horizon 1 and the latest horizon 36, so a given target month sits at a different horizon in each file containing it. Metrics computed per origin keep those apart; metrics pooled across origins mix them.

Each file holds 2,333,448 rows — 36 months × 64,818 cells — and five columns.

Table 4: Columns in every file.

Column	Contents
`month_id`	Month
`priogrid_id`	Grid cell
`pred_lr_ged_sb`	Forecast state-based fatalities
`pred_lr_ged_ns`	Forecast non-state fatalities
`pred_lr_ged_os`	Forecast one-sided fatalities

Units are fatality counts, untransformed — not logged, not scaled. A value of 2.4 means 2.4 fatalities. Values are continuous, not integers.

Nine models × thirteen origins = **117 files, about 2 GB** in total.

6 Consuming the Delivery

Observed Values Are Not in This Folder, and Do Not Need to Be

The evaluation joins forecasts to observed fatalities on `month_id` and `priogrid_id`. Those observations are not included here, and Sonja's ensemble-construction tool does not need them to be: it fetches actuals itself from a viewser queryset named in its own configuration, and also accepts an actuals dataframe passed in directly. They can equally be pulled from the dataFactory.

Three things about that tool are worth flagging to whoever runs it, taken from reading its current source rather than from memory.

- **The file naming already matches.** It expects `predictions_calibration_{timestamp}_{ts}.parquet` with `ts` running `00`–`12`, which is exactly what is here.
- **Its shipped default configuration is country-month, not PRIO-GRID.** The default keys are `(country_id, month_id)` and the default actuals queryset is a country-level one. The evaluation code itself is indifferent to the level — it merges on whatever keys the configuration names — so pointing it at these models is a configuration change, to `(priogrid_id, month_id)` and a PRIO-GRID actuals queryset, not a code change.
- **It resolves prediction files inside the repository layout**, at `views-models/models/<model>/data/generated/`, rather than from an arbitrary folder. The files here are laid out one folder per model and will need placing accordingly, or the paths adjusting.

If a Research Assistant Runs It Instead

The instruction is: *iterate over the thirteen files per model, join each to the observed values on* `month_id` *and* `priogrid_id`*, and compute point metrics per origin so that horizon effects stay separable.* Parquet reads in one call — `pandas.read_parquet` in Python, `arrow::read_parquet` in R.

Two things to tell them explicitly, because both will otherwise be guessed wrong:

1. forecasts are counts, not logs — no back-transformation;
2. there is no distribution — no distributional scores.

7 Not Included in This Delivery

Table 5: Not included in this delivery.

Item	Where it is instead, or why not
Observed values	Fetched by the ensemble tool, or from the dataFactory (Section 6)
Evaluation metrics	None computed — see Section 8
Training logs	Not shipped. `calibration_log.txt`, which the ensemble tool can optionally read to resolve run timestamps, is among them; timestamps can be passed explicitly instead
Other partitions	Calibration only — no validation partition, no true forecasts
Model artefacts	No weights, no checkpoints

8 Caveats and Notes

No model here has a predictive distribution. These are point forecasts — one number per cell-month, no draws, no quantiles, no intervals. All nine are configured as deterministic predictors, so a different export would not recover a distribution; it was never produced. Nothing that scores a *single model's* uncertainty — CRPS of its predictive density, interval coverage, density calibration — can be computed from these files.

One qualification, since it would otherwise look like a contradiction in the output: the ensemble tool can still produce a CRPS. It does so by treating the member models' point forecasts as an empirical sample and scoring the spread across models. That is a property of the ensemble, not of any model in it, and it is not the same quantity as a CRPS computed from a model's own posterior. Worth knowing before the two are compared.

Nothing here has been evaluated. I checked only that the files are not broken: no negatives, no missing values, no duplicate cell-months, the expected row counts, and totals within the same order of magnitude as observed. That rules out a failed back-transformation or a collapsed model and establishes nothing else.

It is nine models, not eleven. Two others were set up to draw 100 samples per forecast, making them probabilistic counterparts of `dancing_monkey` and `dark_necessities`. A point delivery needs one value per cell, and at one sample their configurations are byte-identical to those siblings — the same model twice. Running them as intended is blocked by memory: 100 samples for 64,818 cells across thirteen origins needs far more RAM than any machine we have.

One observation will dominate any squared-error metric. December 2021, cell 148759 has **113,395** observed fatalities; the next largest anywhere in the window is 5,395. That month falls in only one of the thirteen origins (`..._12`), where that single cell is about 41% of observed fatalities. I do not know whether it is a real event or a data error. Either way it is better settled before the numbers are produced than explained afterwards.

The software is pinned; the data is not. The title page records the three versions that determine the output — the model library, the framework and the data backend — along with the `views-models` commit the configurations were read at. Between them those fix the code. They do not fix the *input*: the data backend served this run from a live store with no snapshot identifier, and the provenance record written beside the fetched data shows drift detection did not run. So re-running these nine models at these versions would reproduce the method, and would not guarantee reproducing the numbers, because the underlying observations may have been revised in the meantime. Worth knowing before anyone treats a re-run as a check.

Column naming matches the tool. These files use `pred_lr_ged_sb`, which is the ensemble tool's own default `target_column`. The September delivery used `pred_lr_sb_best` instead, so if anything needs renaming when the two are compared it is that one, not this. Renaming either takes minutes and needs no re-run.

Comparison with the September delivery. You were sent three variants per model then —

a mean, a 95th percentile and a conditional mean — each computed from that model family's predictive distribution. These nine have no distribution, so only the mean has an equivalent here. If the evaluation work has been built on one of the other two, the two deliveries are not like-for-like. A correction while I am here: my September note said the full distributions had been retained so alternative summaries could be produced without re-running. They were not kept.